Lập trình mạng · Chương 4

# Web service: REST, tài nguyên, phương thức HTTP và JSON

TS. Đặng Ngọc Hùng

# REST dùng chính URI, phương thức và mã trạng thái của HTTP

1. SOAP: mọi thao tác là một **POST** tới cùng một địa chỉ; thao tác, tham số, lỗi đều nằm trong SOAP envelope
2. HTTP đã có sẵn: **URI** gọi tên đối tượng, **phương thức** nói thao tác, **mã trạng thái** báo kết quả
3. REST dùng chính các thành phần đó; thân message thường là **JSON**

> **Câu hỏi của phần:** thiết kế API REST cho dịch vụ tra cứu thế nào, và client xử lý kết quả ra sao?

# Mỗi khóa là một tài nguyên; phương thức là thao tác, mã trạng thái là kết quả

`http://127.0.0.1:9199`**`/entries/hanoi`**

endpoint của server — tài nguyên: mục hanoi trong tập entries

| phương thức | thao tác | mã trạng thái | idempotent |
|---|---|---|---|
| **GET** | tra cứu | 200 kèm JSON · 404 không có khóa | có (không đổi dữ liệu) |
| **PUT + JSON** | tạo hoặc thay mục | 201 tạo mới · 200 thay | có (ghi đè cùng giá trị) |
| **DELETE** | xóa mục | 204 đã xóa · 404 không có khóa | có (vẫn là không có) |
| **khác** | không hỗ trợ | 405 · khóa, JSON sai: 400 | |

**REST**: URI định danh **tài nguyên**, **phương thức HTTP** là thao tác, **mã trạng thái HTTP** báo kết quả; server **stateless**.

# Cùng một lời gọi tra cứu: SOAP và REST

| | SOAP | REST |
|---|---|---|
| request | `POST /lookup`<br>thao tác, tham số trong envelope XML: 205 byte | `GET /entries/hanoi`<br>thao tác là phương thức, khóa trong URI, không có thân |
| response | `200 · text/xml`<br>envelope XML: 250 byte | `200 · application/json`<br>JSON: 53 byte |
| báo lỗi | nằm trong Body (SOAP Fault) | mã trạng thái HTTP: 404 không có khóa, 400 yêu cầu sai |
| hợp đồng | bắt buộc: WSDL | không bắt buộc; có thể mô tả bằng OpenAPI |

> REST có message nhỏ hơn và dùng chính HTTP; đổi lại, hợp đồng không bắt buộc.

# Server REST bằng HttpServer của JDK: URI cho khóa, phương thức cho thao tác

```java
server.createContext("/entries/", RestServer::handle);  // mọi URI bắt đầu bằng /entries/
String key = ex.getRequestURI().getPath().substring("/entries/".length());  // URI định danh tài nguyên
switch (ex.getRequestMethod()) {  // phương thức HTTP là thao tác
    case "GET" -> {
        String value = DATA.get(key);
        if (value == null) {
            send(ex, 404, Map.of("error", "not found"));
```

- `createContext`: mọi request có URI bắt đầu bằng /entries/ tới cùng một handler
- Khóa lấy từ **URI**, thao tác chọn theo **phương thức HTTP**
- Mỗi nhánh trả **mã trạng thái** kèm thân **JSON**; PUT ghi đè nên gọi lại cho cùng trạng thái

# Client HttpClient: đọc mã trạng thái trước, rồi mới đọc JSON

Java

```
HttpRequest request = HttpRequest.newBuilder(URI.create(BASE + key))
        .method(method, json == null ? BodyPublishers.noBody() : BodyPublishers.ofString(json))
        .header("Content-Type", "application/json")
        .build();
HttpResponse<String> response = CLIENT.send(request, BodyHandlers.ofString());
int status = response.statusCode();  // đọc mã trạng thái trước, rồi mới đọc thân
```

Kết quả chạy

```
GET    /entries/hanoi    -> 200 {"key":"hanoi","value":"Thủ đô của Việt Nam"}
GET    /entries/atlantis -> 404 {"error":"not found"}
PUT    /entries/sapa     -> 201 {"key":"sapa","value":"Thị trấn vùng cao"}
PUT    /entries/sapa     -> 200 {"key":"sapa","value":"Thị trấn vùng cao"}
DELETE /entries/sapa     -> 204
DELETE /entries/sapa     -> 404 {"error":"not found"}
POST   /entries/sapa     -> 405 {"error":"method not allowed"}
```

# Báo lỗi bằng mã trạng thái; chỉ tự gọi lại phương thức idempotent

1. Trả 200 kèm thân báo lỗi: client và cache coi là **thành công**
2. **PUT**, **DELETE** idempotent: gọi lại để lại cùng trạng thái, dù mã trạng thái có thể khác (201 rồi 200; 204 rồi 404)
3. **POST** không idempotent: gọi lại có thể tạo hai bản ghi

> **Nguyên tắc:** client chỉ tự gọi lại các phương thức idempotent: GET, PUT, DELETE.

# Câu hỏi 1

## Với khóa không tồn tại, server REST trả mã 200 kèm thân {"error": "not found"}. Điều gì có thể sai?

A. Client chỉ kiểm tra mã trạng thái và cache trên đường đi đều coi đây là kết quả thành công.

B. Không sai gì, vì thân JSON đã ghi rõ lỗi.

C. HttpClient ném ngoại lệ vì thân không đúng định dạng.

D. Server phải chuyển sang SOAP để báo được lỗi.

# Câu hỏi 1

## Với khóa không tồn tại, server REST trả mã 200 kèm thân {"error": "not found"}. Điều gì có thể sai?

A. Client chỉ kiểm tra mã trạng thái và cache trên đường đi đều coi đây là kết quả thành công.

B. Không sai gì, vì thân JSON đã ghi rõ lỗi.

C. HttpClient ném ngoại lệ vì thân không đúng định dạng.

D. Server phải chuyển sang SOAP để báo được lỗi.

Mã trạng thái là cách HTTP báo kết quả cho mọi thành phần trên đường đi. Với khóa không tồn tại, mã đúng là 404; trả 200 khiến client và cache hiểu là thành công.

# Câu hỏi 2

**Client gửi request nhưng kết nối hỏng trước khi nhận được response. Với những phương thức nào, client có thể tự gọi lại mà không cần cơ chế bổ sung?**

A. POST và PUT.

B. Chỉ POST.

C. GET, PUT và DELETE.

D. Mọi phương thức, vì HTTP chạy trên TCP.

# Câu hỏi 2

**Client gửi request nhưng kết nối hỏng trước khi nhận được response. Với những phương thức nào, client có thể tự gọi lại mà không cần cơ chế bổ sung?**

A. POST và PUT.

B. Chỉ POST.

C. GET, PUT và DELETE.

D. Mọi phương thức, vì HTTP chạy trên TCP.

GET, PUT, DELETE là idempotent: thực hiện lại để lại cùng trạng thái. POST không idempotent, gọi lại cần idempotency key.

# Tổng kết

1. **REST**: **tài nguyên** định danh bằng **URI**; **phương thức HTTP** là thao tác; **mã trạng thái HTTP** báo kết quả
2. Thân message thường là **JSON**; server **stateless**, mỗi request mang đủ thông tin
3. GET, PUT, DELETE idempotent, POST thì không; báo lỗi bằng mã trạng thái

**Từ khóa cần nhớ** REST · tài nguyên · URI · phương thức HTTP · mã trạng thái HTTP · JSON · stateless

# Mở rộng

### Chủ đề nâng cao

- OpenAPI: mô tả hợp đồng cho API REST
- Quản lý phiên bản API

### Bài tập mở rộng

Thêm GET /entries?prefix=ab trả danh sách khóa có tiền tố; mô tả API bằng một tệp OpenAPI ngắn.

Tiếp theo

# gRPC: Protocol Buffers và lời gọi unary